v0 (1)

↗ Tasks	Design System

Design System Backend — Blog Simple (v1)
- 1. Overview and Objectives
- 2. Arquitecture
- 3. System Architecture
  - Components
- 4. Data Modeling (ERD)
  - Entity Relationship Diagram
  - Notes
- 5. API Contracts (Main Endpoints)
  - Authentication
  - Users
  - Posts
  - Comments
  - Categories
- 6. Key Sequence Diagrams
  - 6.1 Create Post
  - 6.2 Add Comment (Nested Reply)
  - 6.3 Like / Unlike
  - 6.4 Follow User
- 7. Security and Authentication
- 8. Indexes, Queries, and Optimizations
- 11. Future Extensions

Design System Backend — Blog Simple (v1)

Author: Sahira Tejada

Version: 1.0

Date: October 7, 2025

Project: Blog Simple Backend

Technologies: FastAPI, SQLAlchemy (ORM), PostgreSQL, Alembic (migrations),JWT for authentication,.

---

1. Overview and Objectives

This backend powers a simple blogging platform that supports:

- User registration, authentication, and profile management
- Post creation, editing, listing, and deletion
- Comments and nested replies (up to 2 levels)
- Follow/unfollow between users
- Likes on posts
- Filtering and searching posts by category

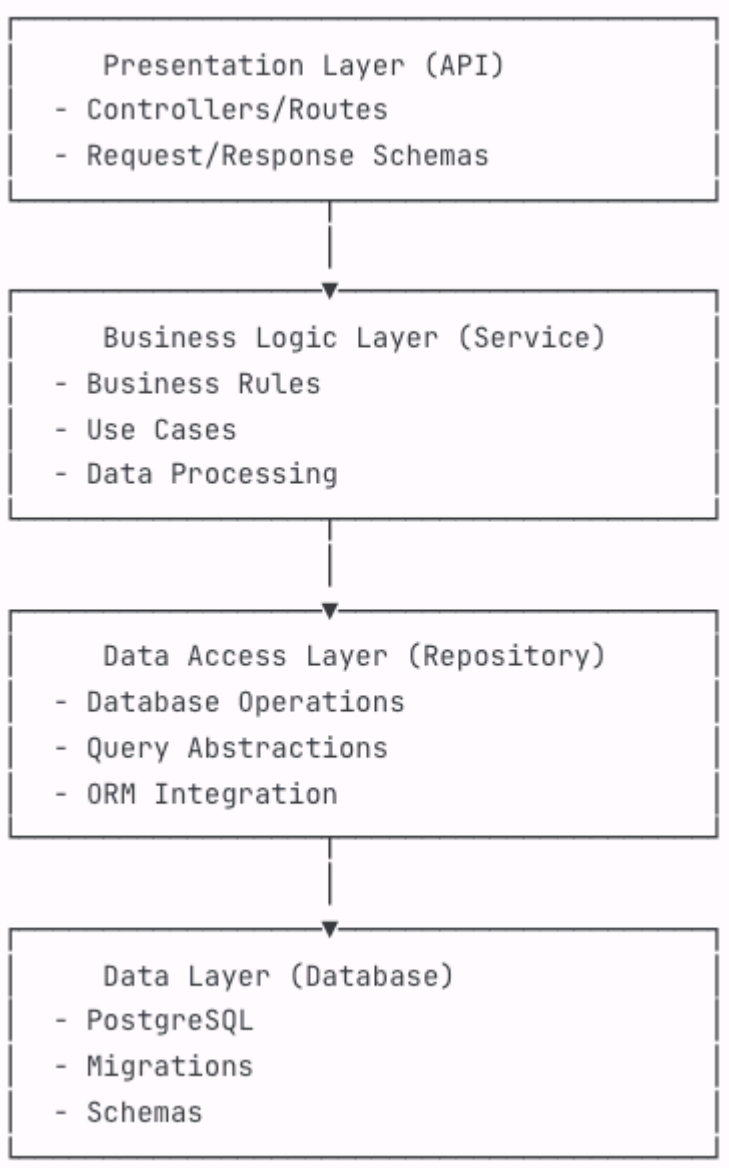
Non-functional goals:

Fast response times, secure API design, scalability, maintainable architecture, and strong test coverage.

---

2. Arquitecture

The project follows a layer arquitecture of 4 levels



3. System Architecture

Components

- **API Layer (FastAPI):** Routers, dependency injection, input/output validation via Pydantic.
- **Service Layer:** Business logic for users, posts, comments, and categories.
- **Data Layer (SQLAlchemy):** Models, sessions, and repositories.
- **Migrations (Alembic):** Schema evolution.
- **Cache (Redis):** Optional caching for timelines, like counts, and category lists.

- **Background Worker (Celery):** Optional for async notifications or analytics.
- **Testing:** Pytest, factories, and fixtures for integration and unit tests.

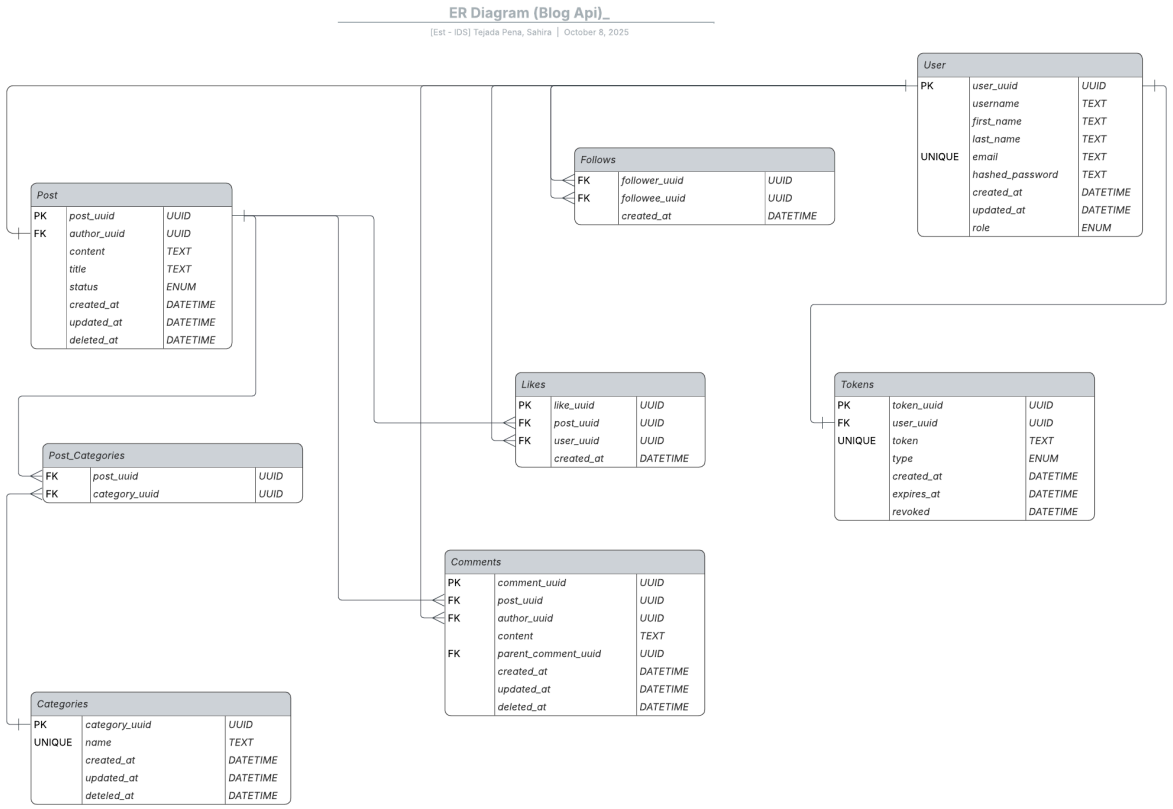
---

4. Data Modeling (ERD)

Entities include:

- `users`
- `posts`
- `comments`
- `categories`
- `post_categories` (many-to-many)
- `likes`
- `follows`
- `tokens`

Entity Relationship Diagram



Notes

- `comments.parent_comment_id` enables nested replies.
- Limit nesting to 2 levels for simplicity.
- Add indexes on foreign keys and timestamps for performance.
- UUIDs are used as primary keys for all entities.

---

5. API Contracts (Main Endpoints)

Authentication

Method	Endpoint	Description
POST	`/auth/register`	Register new user
POST	`/auth/login`	Login and get JWT
GET	`/auth/me`	Get current user info

Users

Method	Endpoint	Description
GET	`/users/{username}`	View public profile
POST	`/users/{username}/follow`	Follow user
POST	`/users/{username}/unfollow`	Unfollow user
GET	`/users/{username}/followers`	List followers
GET	`/users/{username}/following`	List followed users

Posts

Method	Endpoint	Description
GET	`/posts`	List posts (with filters: `category`, `author`, `search`)
POST	`/posts`	Create post (auth required)
GET	`/posts/{id}`	Get post details
PUT	`/posts/{id}`	Update own post
DELETE	`/posts/{id}`	Delete own post

Method	Endpoint	Description
POST	`/posts/{id}/like`	Like/unlike post

Comments

Method	Endpoint	Description
POST	`/posts/{post_id}/comments`	Add comment
GET	`/posts/{post_id}/comments`	List comments (nested view)
DELETE	`/comments/{id}`	Delete comment (own/admin)

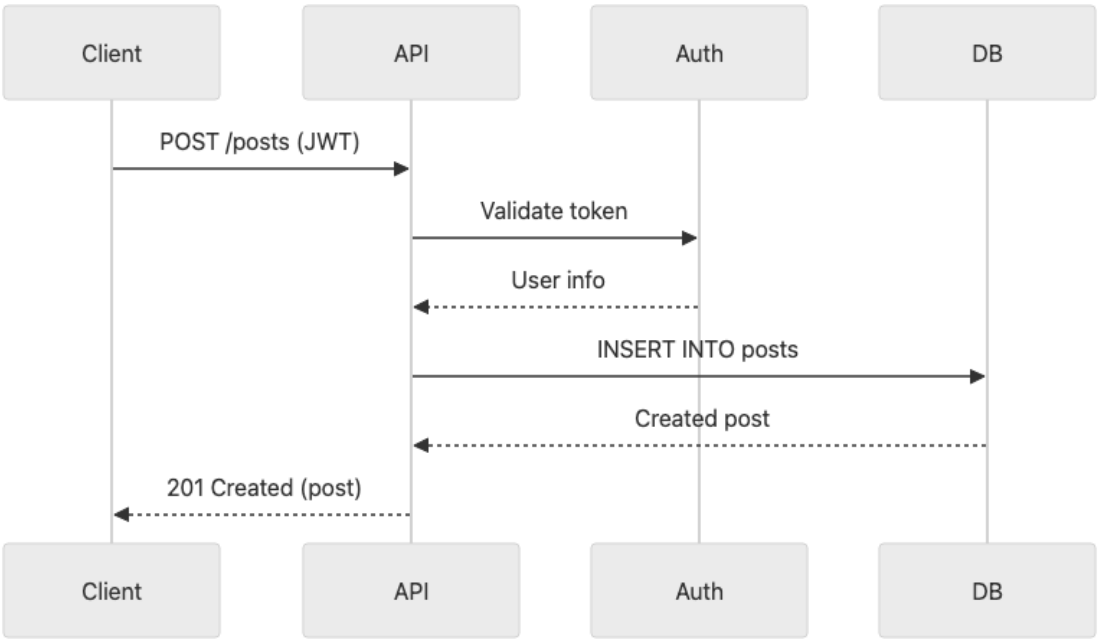
Categories

Method	Endpoint	Description
GET	`/categories`	List categories
POST	`/categories`	Create category (admin only)
PUT	`/categories/{id}`	Update category (admin only)
DELETE	`/categories/{id}`	Delete category (admin only)

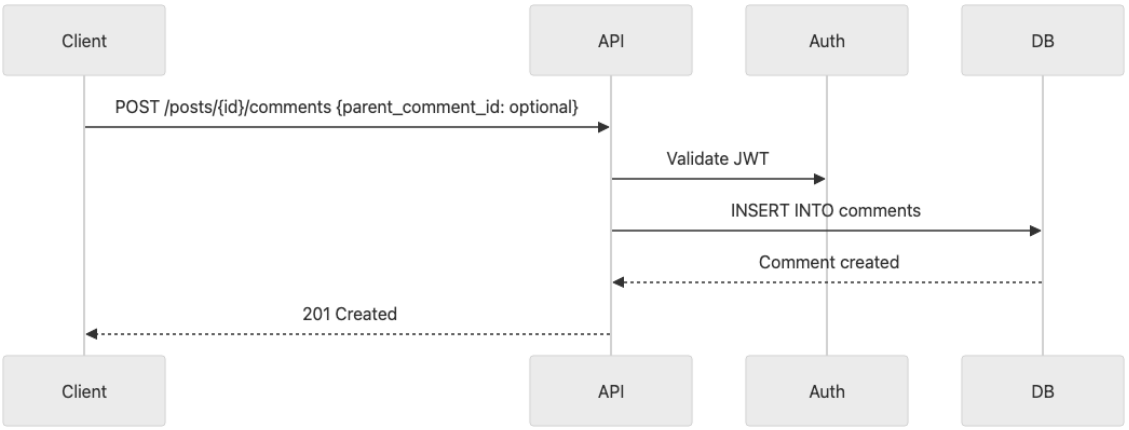
---

6. Key Sequence Diagrams

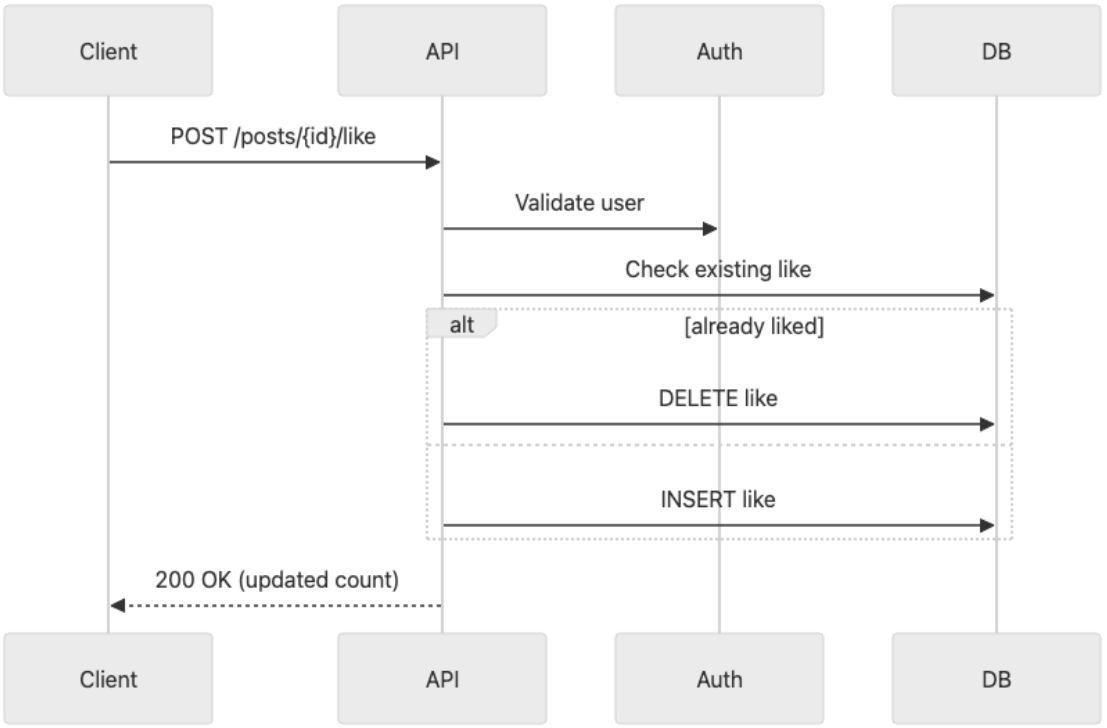
6.1 Create Post



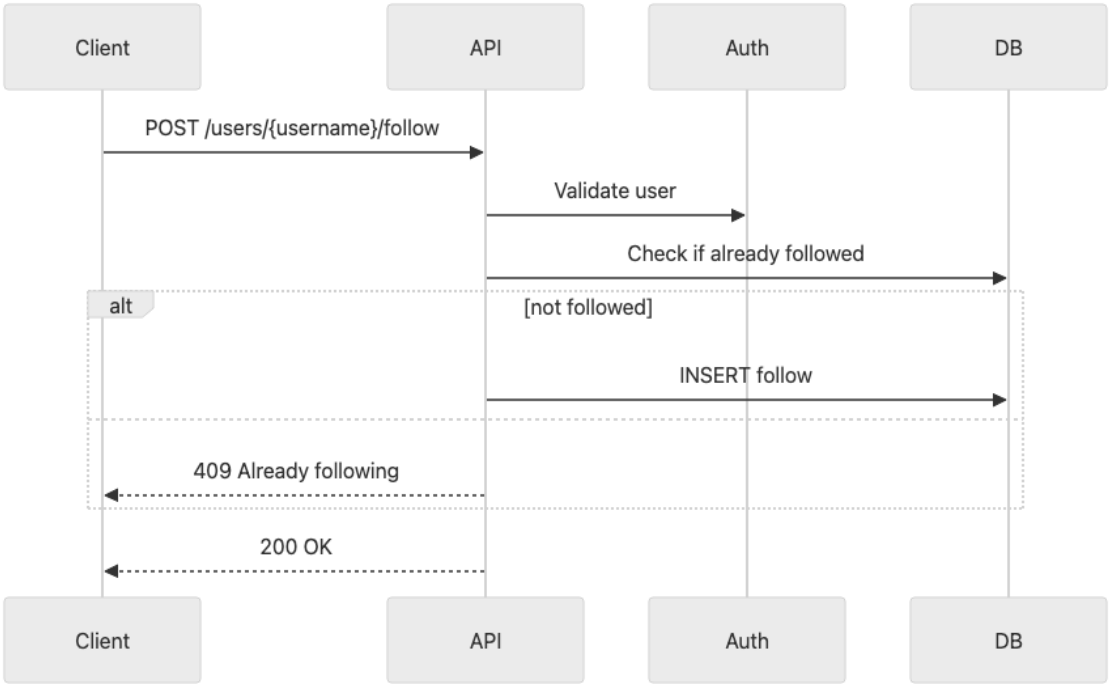
6.2 Add Comment (Nested Reply)



6.3 Like / Unlike



6.4 Follow User



7. Security and Authentication

- Use HTTPS for all endpoints.
- Authentication vwith JWT tokens.
- Passwords hashed using `bcrypt` or `argon2`.
- Role-based permissions:
  - **Guest:** View posts/comments
  - **User:** Create/edit/delete own posts/comments, like/follow
  - **Admin:** Manage all resources
- Rate limiting for login and like actions.
- Input validation and sanitization to avoid injection.
- Ownership checks for modifying content.

---

8. Indexes, Queries, and Optimizations

Recommended Indexes:

- `posts.created_at`, `posts.author_id`
- `categories.slug`
- `comments.post_id`, `comments.parent_comment_id`
- `likes.user_id`, `likes.post_id` (unique)
- `follows.follower_id`, `follows.followee_id`

Conventions:

- Use snake_case for filenames.
- Type hints required for all functions.
- Pydantic schemas mirror SQLAlchemy models for I/O.
- Use dependency injection for DB sessions and auth.

---

11. Future Extensions

- Role-based access control with fine-grained permissions.

- Real-time notifications using WebSockets.
- Post drafts and versioning.
- Activity feed (followers' posts).
- Integration with ElasticSearch for advanced text search.
- Analytics and caching via Celery tasks.

Optimizations:

- Denormalized counters for likes/comments in `posts` table.
- GIN + `tsvector` search for text queries.
- Cursor-based pagination for large datasets.
- Optional Redis caching for trending posts and categories.